Adaptive, Intelligent Multi-Agent System for Autonomous Software Planning, Code Generation, and Self-Debugging

R26-SE-029

Project Proposal Report

Koshigawarman Y. – IT22273444

B.Sc. (Hons) in Information Technology specialized in
Software Engineering

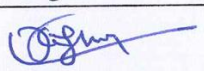
Department of Information Technology

Sri Lanka Institute of Information Technology, Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Koshigawarman Y.	**IT22273444**	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Ms. Sanjeewi Chandrasiri	2026 / 03 / 20	

ABSTRACT

Modern software development is increasingly reliant on AI-generated code, yet approximately 30% of these outputs contain execution-critical bugs or hallucinations. This research proposes an Intelligent Multi-Agent System that automates the backend development lifecycle through specialized autonomous agents. While existing solutions offer static code suggestions, they lack a secure, automated runtime validation loop. This project introduces a Human-on-the-Loop framework where a dedicated Testing Agent executes code within an isolated Docker Sandbox. By using Reflexive Critique and Contextual Feedback, the system captures runtime errors and generates structured bug reports to trigger autonomous self-correcting. Utilizing Node.js, Python, and LangGraph, the system aims to reduce Mean Time to Repair (MTTR). The social impact includes lowering technical barriers for startups, while the commercial value lies in delivering "pre-validated" production-ready code.

Keywords: ***Multi-Agent Systems***, ***Autonomous Testing***, ***Docker Sandboxing***, ***Self-Healing Software***, ***LLM***.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me throughout this final year research project.

First, I am deeply thankful to my supervisor, **Ms. Sanjeewi Chandrasiri**, for her invaluable guidance, encouragement, and technical expertise which were vital to the successful completion of this research. I also extend my thanks to the **CDAP panel** for their constructive feedback and advice during the project evaluations.

Special thanks to my co-supervisor, **Ms. Karthiga Rajendran**, for her professional insights and advice that helped streamline my work. I would also like to acknowledge my **research project group members** for their collaboration and mutual support throughout the development process.

Finally, I am forever grateful to my **parents and friends** for their constant motivation and support, which made the completion of this project possible

TABLE OF CONTENT

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI – Artificial Intelligence

API – Application Programming Interface

B2B – Business-to-Business

CDAP – Comprehensive Design & Analysis Project

CRUD – Create, Read, Delete, & Update

DSR – Design Science Research

FR-TA – Functional requirement – Testing Agent

JSON – JavaScript Object Notation

LLM – Large Language Model

MAS – Multi-Agent System

MTTR – Mean Time To Repair

QA – Quality Assurance

SaaS – Software as a Service

SDG – Sustainable Development Goal

SDLC – Software Development Life Cycle

SLIIT – Sri Lanka Institute of Information Technology

LIST OF APPENDICES

1. INTRODUCTION

1.1. Background and Context

The Crisis of Reliability in AI-Generated Code

The modern software industry is under immense pressure to deliver features faster than ever before. This has led to the rapid adoption of Artificial Intelligence (AI) to automate code generation. While these tools have significantly reduced the time it takes to write initial drafts of software, they have introduced a new crisis: a lack of execution reliability. Traditional AI assistants provide code suggestions based on patterns but do not understand the underlying system architecture or security requirements of a specific environment. Statistics from recent research indicate that nearly 30% of AI-generated code fails to run correctly on the first attempt [1].

Critical Risks for Backend Engineering

This reliability gap is particularly dangerous for backend engineering. In a backend service, a single unverified script—such as an incorrect database query or a flawed authentication check—can lead to severe consequences, including data corruption or system-wide crashes. This problem affects everyone from solo developers to large-scale SaaS enterprises, where any downtime translates directly into financial loss. Despite the convenience of current AI tools like GitHub Copilot, they remain "passive" because they lack the ability to verify their own output in a live environment.

The Role of the Testing Agent as a Gatekeeper

To address this, our project introduces a team of autonomous agents that mimic a professional development squad. My specific component, the Testing Agent, acts as the "Gatekeeper" of the system. It is designed to take the code produced by the Coding Agent and subject it to a series of rigorous checks before it is ever allowed to reach a production environment [9]. By utilizing a Docker-based sandbox, we create a secure, isolated "bubble" where code can be executed safely [7]. This research is a direct contribution to SDG 9 (Innovation and Infrastructure) and aligns with the Software Engineering Research Cluster by creating a verifiable, autonomous pipeline for the Software Development Life Cycle (SDLC).

1.2. Literature Review

The Evolution Toward Agentic AI The current state of research suggests that we are moving away from simple AI-guided tools toward "Agentic AI"—systems that can perform multi-step reasoning across the entire SDLC [11]. A foundational study by Garlapati et al. (2024) demonstrates that LLM-powered agents can achieve 100% code coverage by iteratively writing and refining unit tests [7]. While this proves that AI is capable of generating tests, it also highlights a major oversight in traditional research: the assumption that the testing environment itself is safe and properly configured.

Security and Fault-Tolerant Sandboxing This security concern is a recurring theme in recent scholarly work. Researchers like Yan (2025) have argued that since AI can "hallucinate" malicious or destructive system commands, it is essential to use Fault-Tolerant Sandboxing [9]. This isolation prevents the AI from accidentally deleting host files or leaking sensitive configuration data [8]. By isolating the execution layer, we can allow the AI to run "headless loops" of testing without human supervision, which is a prerequisite for true autonomy.

Learning Through Contextual Feedback However, isolation is only half of the solution. The other half involves learning from failures. The work of Liu et al. (2013) and Li et al. (2026) emphasizes that the most efficient way to repair software is by using "contextual feedback" from the runtime environment [10]. My component builds on these two pillars—security and feedback—by combining a Transactional Sandbox with a Diagnostic Layer. This allows the Testing Agent to not only catch a crash but also generate a structured report that tells the system exactly how to fix the bug [4]. This combination of execution and self-correction represents the next evolution in autonomous software engineering.

1.3. Research Gap

The Need for Closed-Loop Validation Despite the high number of AI tools available today, there is a clear "Research Gap" when it comes to the closed-loop execution of code. Most existing products only focus on the generation of source code but fail to provide a mechanism for real-time, autonomous validation. This forces human developers to spend hours manually debugging AI-generated scripts, which defeats the purpose of automation [9].

Inconsistencies in Feedback and Safety Furthermore, current frameworks lack a standardized "Handshake" between the execution environment and the repair logic. Most systems provide simple "Pass/Fail" results, which are not detailed enough for an AI to use for self-correction [4]. There is also a significant lack of integrated tools that provide host-system protection while maintaining high-speed testing cycles. Our research fills this gap by introducing an Autonomous Diagnostic Layer that turns raw terminal output into structured "Lessons" [11]. This novelty ensures that the system is not just generating code, but is actively ensuring that the code is functionally correct and safe.

Research Gap Area	Existing Research & Tools	Proposed Testing Agent & Sandbox
Code Verification	Only checks if the code looks right (Static Analysis)	Checks if the code actually runs (Dynamic Execution)
System Safety	Runs code directly on the machine, which is risky	Run code in a Secure Docker Sandbox
Malicious Code	Can accidently run dangerous commands like rm-rf	Uses safety scan to block harmful commands before they run
Error Feedback	Gives simple, confusing error messages	Turns crashes into Structured JSON Reports the AI can understand
Human Effort	Developers must manually fix AI mistakes	The system fixes its own bugs using an autonomous feedback loop

Table 1

2. OBJECTIVES

2.1. Main Objective

To develop an autonomous Testing Agent that ensures the functional correctness of AI-generated services through Sandboxed Execution and structured feedback loops.

2.2. Specific Objectives

To implement a Static Safety Scan to intercept destructive commands using policy-based filtering.

To design a Docker-based Sandbox that isolates the execution of Node.js services from the host system.

To build an agent capable of generating Jest and Supertest scripts to achieve at least 80% code coverage autonomously.

To develop a Diagnostic Layer that parses terminal crash logs into structured JSON Bug Reports for autonomous code repair.

3. METHODOLOGY

Research Approach and Strategy

For this project, we have chosen the Design Science Research (DSR) approach. This methodology is ideal for software engineering as it focuses on creating and evaluating a specific "artifact"—in this case, the Testing Agent—to solve a practical, real-world problem. The process begins when the Testing Agent receives a completed set of backend files from the Coding Agent.

Security Protocols and Sandboxing

To ensure security, the first step is a Static Safety Scan, which uses policy-based filtering to identify and block dangerous commands such as `rm -rf` or unauthorized process terminations [9]. Once the code is deemed safe, the agent programmatically spins up a lightweight Docker container. We have chosen Docker over traditional virtual machines because it offers faster startup times and uses fewer system resources while still providing 100% filesystem isolation [8].

Technical Implementation and Diagnostics

Technically, the agent uses autonomic test generators to create functional test scripts using Jest and Supertest. These generators are designed to visit complex boundary conditions and "corner cases" that are often missed by human testers [11]. If the tests fail, the Diagnostic Layer captures the runtime "stack trace" and parses it into a clean JSON format. This report acts as the evidence-based feedback required for the self-correction cycle [9].

Evaluation and Validation Metrics

Validation of the methodology will be conducted through experimental testing. We will measure the Interception Rate (how effectively the system blocks malicious code) and the Correction Efficiency (how many retry loops the system needs to reach a bug-free state) [9]. This structured approach ensures that the final system is not only innovative but also reliable and practically achievable for real-world development teams.

4. HIGH-LEVEL ARCHITECTURE

Overview and Data Flow:

The system operates as a unified multi-agent orchestration. The Data Flow specifically centers on the interaction between the Coding Agent and the Testing Agent:

Input Acquisition: The Testing Agent receives the synthesized source code files directly from the Coding Agent as a completed file-set.

Validation Loop: The code is first scanned for safety, then mounted into the Docker Sandbox.

Reflexive Feedback: If tests fail, the raw execution logs are processed by the Diagnostic Layer into a JSON-formatted report.

Integration: This report is sent back to the Orchestrator to trigger a "Self-Debugging" round for the Coder.

4.1. High-Level Architecture of Component

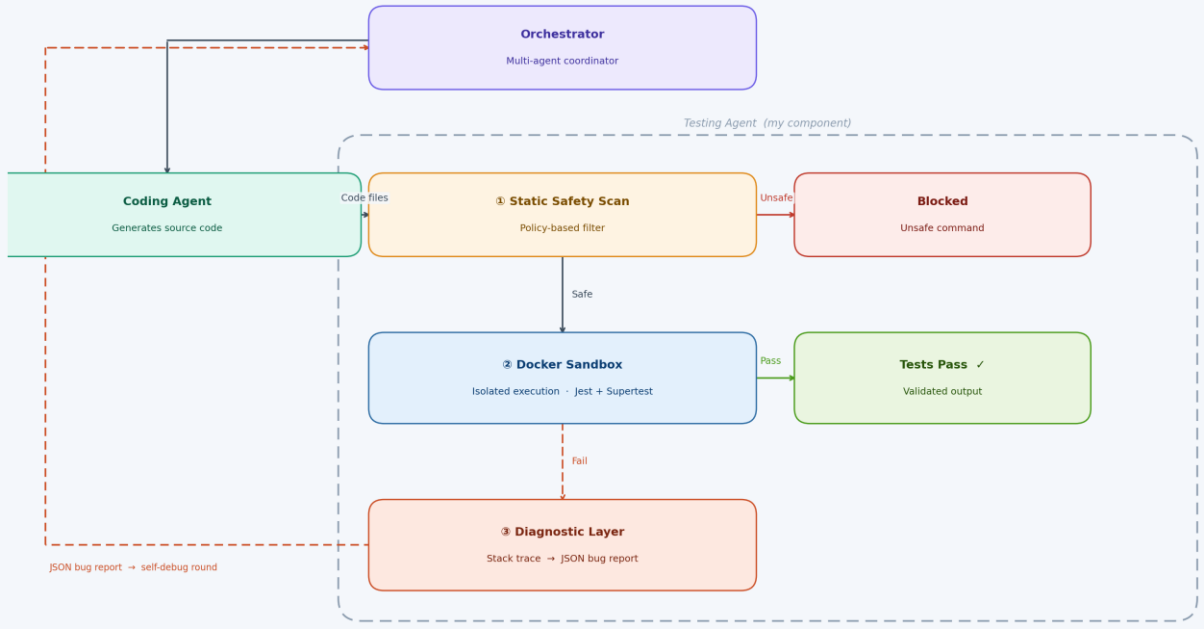


Figure 1

4.2. High-Level Architecture of System

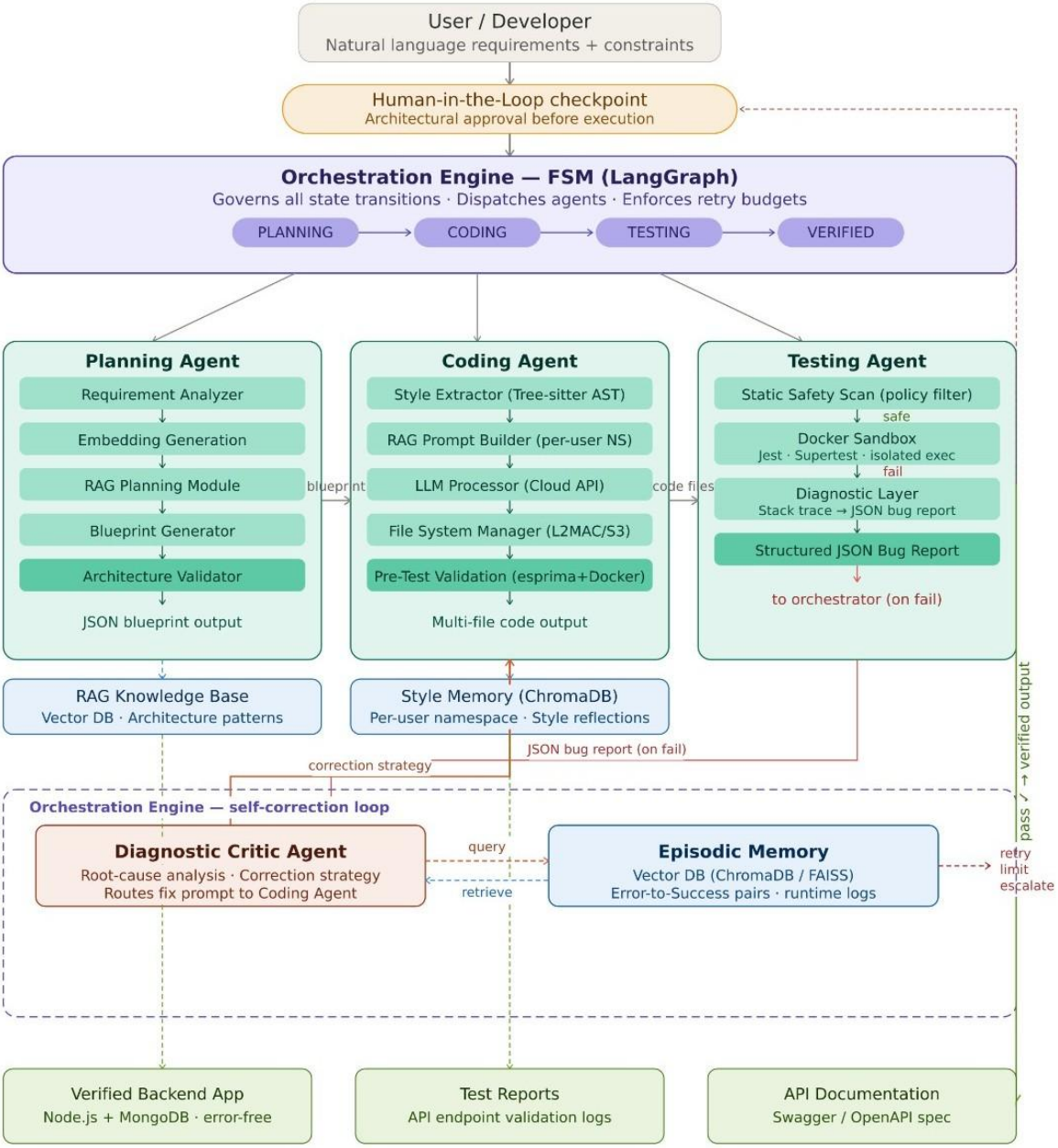


Figure 2

5. USER REQUIREMENTS

5.1. Functional Requirements

- **FR-TA-01:** The system shall accept source code outputs from the Coding Agent as the primary input for validation.
- **FR-TA-02:** The system shall spin up an isolated environment for every API testing cycle.
- **FR-TA-03:** The agent shall generate functional test cases for all detected REST endpoints.

5.2. Non-Functional Requirements

- **Security:** The system must maintain 100% filesystem isolation from the host machine.
- **Performance (Speed):** The validation loop, including container setup and test execution, must complete in under 120 seconds.

6. COMMERCIALIZATION PLAN

Target Market and Customer Persona

Primary market: Sri Lankan software development companies (SMEs and freelancing platforms) and individual backend developers maintaining multiple concurrent client projects. Customer persona: Ravi, a 28-year-old full-stack developer at a Colombo-based startup, maintains 5–7 concurrent client projects, spends 30–40% of his time on repetitive backend code and style refactoring, and wants an API-accessible tool that learns and enforces his personal coding conventions without requiring him to manage any local AI infrastructure.

Value Proposition

Generates production-ready, style-consistent Node.js backends via a simple API call in minutes rather than hours.

The only tool that automatically learns and persistently enforces a developer's personal coding conventions — commercial tools do not do this.

API-accessible and integrable into any IDE, CLI, or CI/CD pipeline with no local setup required.

Open-source core with a transparent, auditable architecture — unlike the black-box commercial tools.

Multi-tenant architecture means teams can share one deployment while each developer retains their own fully isolated style profile.

Revenue Model

Open-source core (MIT licence, GitHub) — free for students, individuals, and small teams; builds community adoption and trust.

Premium Enterprise Edition (paid) — custom style profiles, team shared style libraries, VS Code extension with one-click integration, priority support, and SLA guarantees (LKR 2,500–5,000 per month per team).

Freemium API model — limited monthly generation requests free; higher request quotas, multi-project episodic memory, and auto-Swagger with Docker Compose unlocked via subscription.

Pricing Strategy

One-time individual licence: LKR 15,000. Monthly developer subscription: LKR 1,500. Team bundle (up to 10 developers): LKR 10,000 per month. Training workshops for IT companies: LKR 25,000 per session.

Competitive Advantage

Unlike Cursor, Replit Agent, and Claude Code — which are style-agnostic and treat all developers identically — this agent learns each developer's personal conventions once from their own repository and enforces them permanently across all future projects via a persistent namespaced RAG profile. The multi-tenant architecture means this personalisation scales to many users on shared infrastructure with guaranteed isolation. This combination is uniquely absent from all commercial and academic tools in the current landscape.

7. BUDGET AND JUSTIFICATION

Category	Item	Estimated Cost (LKR)
Hardware Costs	Personal Laptop (Standard Dev Machine) (Already Owned)	0
Software Licenses	Open Source (Docker, Node.js, Jest)	0
Cloud Services & LLM API Tokens	Cloud Hosting (AWS) & OpenAI	9000
Human Efforts	Development & Research Time (Academic Project)	0
Connectivity	High-speed Internet Data	6000
Total		**15000**

Table 2

Cloud Hosting (Sandbox Infrastructure)

To maintain 100% filesystem isolation, the system must run on a cloud-based server that supports Docker. This cost covers the virtual machines used to host the transactional sandboxes during the evaluation and validation phase of the research.

Software Licenses

One of the core strengths of this methodology is the use of Open Source tools. By building on Docker, Node.js, and Jest, we eliminate licensing costs while ensuring the system remains scalable and professionally supported.

Connectivity

A stable, high-speed connection is mandatory for the continuous communication between the Orchestrator and the Cloud LLM APIs. This also facilitates the downloading of Docker images and the deployment of microservices during the integration testing phase.

8. WORK BREAKDOWN STRUCTURE

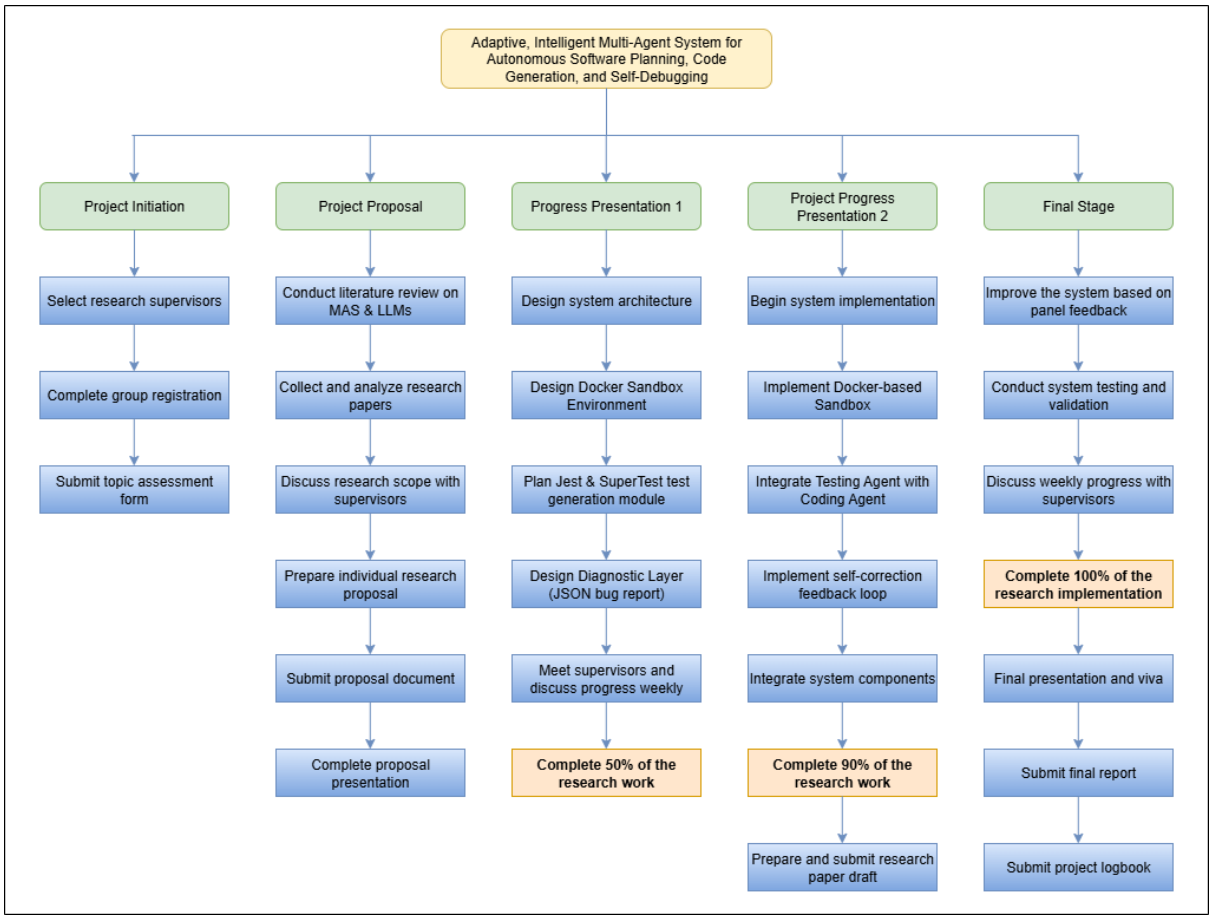


Figure 3

9. GANTT CHART

Figure 4

10. REFERENCES LIST

[1] R. Ramírez-Rueda et al., "Transforming Software Development: Integration of MAS and LLMs," *CONISOFT*, 2024, doi: 10.1109/CONISOFT63288.2024.00013 **[Accessed: Mar. 18, 2026]**.

[2] S. Suri et al., "Autonomous agents, Large Language Models (LLMs), SDLC," *GCAT*, 2023, doi: 10.1109/ASE56229.2023.00174 **[Accessed: Mar. 18, 2026]**.

[3] L. Lin et al., "Process modeling, large language models, multi-agent, hallucination phenomena," *IEEE Trans. Serv. Comput.*, 2025, doi: 10.1109/TSC.2025.3611816 **[Accessed: Mar. 18, 2026]**.

[4] Z. Li et al., "OptDisPro: LLM-Based Multi-Agent Framework for Self-Correction," *IEEE Trans. Smart Grid*, 2026, doi: 10.1109/TSG.2025.3620496 **[Accessed: Mar. 18, 2026]**.

[5] A. Garlapati et al., "AI-Powered Multi-Agent Framework for Automated Unit Test Generation," *GCAT*, 2024, doi: 10.1109/GCAT62922.2024.10923987 **[Accessed: Mar. 18, 2026]**.

[6] H. Jin et al., "From LLMs to LLM-based Agents for Software Engineering: A Survey," *arXiv*, 2025, doi: 10.48550/arXiv.2408.02479 **[Accessed: Mar. 18, 2026]**.

[7] B. Yan, "Fault-Tolerant Sandboxing for AI Coding Agents," *arXiv*, 2025, doi: 10.48550/arXiv.2512.12806 **[Accessed: Mar. 18, 2026]**.

[8] R. Rabin et al., "SandboxEval: Securing Test Environment for Untrusted Code," *arXiv*, 2025, doi: 10.48550/arXiv.2504.00018 **[Accessed: Mar. 18, 2026]**.

[9] R. Konda, "Agentic AI for Software Development: Testing and Deployment," *IJERET*, 2025. [Online]. Available: https://ijeret.org/index.php/ijeret/article/view/415 **[Accessed: Mar. 18, 2026]**, doi: 10.63282/3050-922X.AECTIC-118.

[10] C. Liu et al., "R2Fix: Automatically Generating Bug Fixes from Bug Reports," *ICST*, 2013, doi: 10.1109/ICST.2013.24 **[Accessed: Mar. 18, 2026]**.

[11] N. Takerngsaksiri et al., "Human-in-the-loop LLM-based Agents framework (HULA)," 2025, doi: 10.1109/ICSE-SEIP66354.2025.00036 **[Accessed: Mar. 18, 2026]**.

[12] Y. Li et al., "Test-Agent: Multimodal Agent-based App Automation Testing," 2024, doi: 10.1109/DTPI61353.2024.10778901 **[Accessed: Mar. 18, 2026]**.

11. APPENDICES

Risk Analysis

Risk Description	Impact	Probability	Mitigation Strategy
Sandbox Escape (Malicious code accessing host system)	Critical	Low	Use non-root Docker users, read-only filesystems for core directories, and restricted network access.
Infinite Execution Loops (Tests running forever and wasting resources)	High	Medium	Implement a hard Timeout Policy (example: 60 seconds) that automatically kills the container if execution exceeds the limit.
API Token Depletion (High costs due to diagnostic feedback loop)	Medium	High	Use efficient prompt engineering and "Summarization" of logs to reduce the number of tokens sent to the LLM.
Environment Inconsistency (Test passing in Sandbox but failing on Host)	Medium	Medium	Use standardized Docker Images that mirror the target production environment (example: Node.js Alpine versions).
False Positives in Safety Scan (Blocking safe code accidently)	Low	Medium	Maintain a refined Allow-list of safe system calls and provide an override mechanism for "Verified" scripts.
Resource Exhaustion (Multiple containers crashing the server)	High	Low	Set strict CPU and RAM limits on each Docker container using Docker

			Compose or orchestrator constraints.

Justification of Risk Management

As the Testing Agent acts as the final gatekeeper before the code is delivered, the primary focus is on Security and Reliability. The Static Safety Scan acts as the first line of defense, while the Docker Sandbox serves as the fail-safe. By implementing strict timeouts and resource caps, we ensure that the system remains stable even if the Coding Agent generates inefficient or recursive code.